Run Guide: build B1 Track B with the Claude Code CLI

Build the B1 project step by step by prompting Claude Code, and check every step yourself
Modern AI Pro · Level 2 · Track B · RAG Design · Owner: Shreenath Shetty

Version 5 · 2026-10-18 · all 12 steps done · public on GitHub · 30-case test plan

What this is	A simple, step-by-step script for building the B1 project by giving Claude Code one prompt per step
You need	Claude Code CLI, `git`, `uv`, Python 3.12 (installed by `uv`), the class `.env` key
Base folder	`~/Desktop/AI-Architect_L2_Aug_2026/project_10082026`
Companion docs	SOP PDF (the full detail) and the Flow Deck, both in `b1-tracking/track-b/docs/`
Status	All 12 steps done and tested; published to GitHub
GitHub	https://github.com/shreenathacc22/L2-RAG-B1-track-b-kafka-version-aware (public): `main`, `track-b-kafka-version-aware`, `rag-deployment`

1. How to use this guide

Every step follows the same pattern:

1. **Paste the prompt** into Claude Code exactly as written. It's in a dark box.
2. **Let Claude work.** Approve the commands it asks to run.
3. **Check it yourself** with the commands under "You check". Claude reports results, but these commands are your proof.
4. **Move on** only when the check matches "Expect".

If a check doesn't match, tell Claude what you saw, for example *"b1_check.py says FAIL (2) on kafka-005, fix it"*.

2. Start a session

```
claude --version                                          # Claude Code is installed
cd ~/Desktop/AI-Architect_L2_Aug_2026/project_10082026  # ALWAYS start here
claude                                                    # opens the session in this folder
```

Install Claude Code from https://docs.claude.com/en/docs/claude-code/setup if `claude --version` fails.

Useful keys and commands inside a session:

Do this	To
`Esc`	Stop Claude mid-step (nothing is lost)
`!` + command, e.g. `!git log --oneline -3`	Run a shell command yourself without leaving the session
`/clear`	Start a fresh conversation (files are kept)

Do this	To
`/resume` , or start with `claude -c`	Continue an earlier session
`/exit`	Leave

First prompt of every session (sets the ground rules):

```
Work in ~/Desktop/AI-Architect_L2_Aug_2026/project_10082026. The code is in track-b-kafka/
(branch track-b), the evidence and docs are in b1-tracking/track-b/. Read
b1-tracking/track-b/SOP.md first. Never print the .env key. Don't push to GitHub
unless I ask. Save every run's log under b1-tracking/track-b/runs/<stepNN-name>/.
```

3. Shortcut: rebuild Steps 0–12 with one prompt

```
Run b1-tracking/track-b/tools/recreate_track_b.sh with
VENV=$PWD/mai-practitioner-labs/.venv into a new folder track-b-rebuild, then tell me
whether every fingerprint says MATCH, what the BEFORE superseded rate is, and whether
the Chroma, OFF/ON and AFTER checks passed.
```

You check: the last lines say `MATCH` seven times, BEFORE is `48.8%`, AFTER is `0.0% · 9/9 · 7/7 · 3/3`, and the run ends with `Done · Steps 0–12`. **Clean up:** `!rm -rf track-b-rebuild track-b-rebuild.logs`

4. Step by step

Step 0: get the code

```
Clone https://github.com/balajivis/mai-practitioner-labs.git into track-b-kafka, check out
commit 6b918c2, create branch track-b, and copy ./.env into it with chmod 600. Confirm
.env is gitignored.
```

You check: `!git -C track-b-kafka branch --show-current` → `track-b`

Python environment

```
In track-b-kafka, create a Python 3.12 venv with uv at .venv and install requirements.txt
plus the Intel-Mac pins "torch==2.2.2" "numpy<2" "transformers<5" "sentence-transformers<6".
Then check that torch, chromadb, sentence_transformers and openai import.
```

You check: `!track-b-kafka/.venv/bin/python -c "import torch; print(torch.__version__)"` → `2.2.2`

Step 2: the unchanged lab works

```
Activate track-b-kafka/.venv and run simple_rag unchanged: rag.py ingest, rag.py ask
"how do I tow my car?", rag.py eval. Save the three logs to
b1-tracking/track-b/runs/step02-baseline-unchanged/ and tell me the eval scorecard.
```

Expect: ingest ~196 vectors, a cited answer, eval `retrieval 2/6 · answer 2/6`. That score is the lab's normal starting point.

Steps 3–4: versioned corpus and manifest

```
Generate the Kafka corpus with b1-tracking/track-b/tools/make_corpus.py into
track-b-kafka/simple_rag/corpus/kafka, copy b1-tracking/track-b/tools/source/b1_check.py
into track-b-kafka/simple_rag/, run b1_check.py (full, with the similarity trap), save the
log under runs/step03-corpus/, and commit corpus + checker on track-b.
```

You check: `!cd track-b-kafka && .venv/bin/python simple_rag/b1_check.py --no-trap | tail -1` → `PASS`
Expect in Claude's report: 16 runbooks, 8 current / 8 superseded, pair similarity 0.867–0.969, old version outranks current for 4/8.

> **Want Claude to design new runbooks instead of reusing ours?** Use: *"Design 8 Kafka runbook subjects for a fictional company (not Meridian or Aventro), each with a current and an old version that share wording but differ in 5–9 facts. No 'superseded' banner, no front-matter. Put metadata in manifest.json (doc, version, effective_date, superseded). Then make b1_check.py pass."* The fingerprints in the SOP then won't match. That's expected.

Step 5: golden set (21 test cases)

```
Generate track-b-kafka/simple_rag/golden_kafka.jsonl with
b1-tracking/track-b/tools/make_golden.py, run b1_check.py --no-trap, then run the two
negative tests from SOP Appendix A.2 (break manifest, break golden set, confirm FAIL,
restore). Save logs under runs/step05-golden/. Do NOT commit the golden set yet.
```

You check: `!shasum track-b-kafka/simple_rag/golden_kafka.jsonl` → `850c67f2eab21aa1a6afabd481e5c3efccb7e0ba` **Then read the 21 questions yourself** (SOP Step 5 table) before freezing.

Step 6: freeze

```
Commit simple_rag/golden_kafka.jsonl ALONE with the message "Freeze B1 golden set before
version-aware retrieval". Show me git log for the golden file and for rag.py.
```

You check: `!git -C track-b-kafka log --oneline -- simple_rag/golden_kafka.jsonl` → exactly one commit.
Rule: from now on, no edits to the golden set without a separate commit that gives the reason.

Step 7: evaluator and BEFORE run (filter OFF)

```
Copy b1-tracking/track-b/tools/source/rag.py and b1_eval.py into track-b-kafka/simple_rag/.
With RAG_CORPUS=simple_rag/corpus/kafka RAG_COLLECTION=kafka, run rag.py ingest --formats md
and then b1_eval.py --off. Save logs and the JSON under runs/step07-before/, and commit
rag.py, b1_eval.py and results/baseline_filter_off.json.
```

You check:

```
!cd track-b-kafka && RAG_CORPUS=simple_rag/corpus/kafka RAG_COLLECTION=kafka .venv/bin/python
simple_rag/b1_eval.py --off | tail -6
```

Expect: `superseded chunks retrieved 41/84 = 48.8%`, superseded 0/9, exact_string 7/7, multi_hop 0/3.

Step 8: version metadata into Chroma

```
Copy b1-tracking/track-b/tools/source/step08/rag.py, b1_check.py and b1_inspect_chroma.py
into track-b-kafka/simple_rag/. First run b1_check.py --no-trap --chroma on the existing
collection and confirm it FAILS (no metadata yet). Then re-ingest with
RAG_CORPUS=simple_rag/corpus/kafka RAG_COLLECTION=kafka, run --chroma again (must PASS),
run b1_eval.py --off and confirm it is identical to the Step 7 baseline, and run
b1_inspect_chroma.py. Save logs under runs/step08-chroma-metadata/ and commit.
```

You check it inside Chroma. All three run from `track-b-kafka/` with the venv active:

```
!cd track-b-kafka && .venv/bin/python simple_rag/b1_check.py --no-trap --chroma | tail -4
```

Expect: `9 chunks from 8 files` / `8 chunks from 8 files`, then `PASS`.

```
!cd track-b-kafka && .venv/bin/python simple_rag/b1_inspect_chroma.py
```

Expect: with the filter OFF, `disk-capacity-2025.md SUPERSEDED` is at #2. With the filter ON, there are no SUPERSEDED rows.

```
!cd track-b-kafka && RAG_CORPUS=simple_rag/corpus/kafka RAG_COLLECTION=kafka .venv/bin/python
simple_rag/b1_eval.py --off | grep "superseded chunks"
```

Expect: still `41/84 = 48.8%`. Metadata alone changes nothing.

Steps 9–10: the filter, behind an OFF/ON switch

```
Copy b1-tracking/track-b/tools/source/step10/rag.py into track-b-kafka/simple_rag/. It adds
version_aware to rag.retrieve(): ON adds where={"superseded": False} to the Chroma query,
OFF keeps the original query. Then, with RAG_CORPUS=simple_rag/corpus/kafka
RAG_COLLECTION=kafka: run b1_eval.py --off and prove it is identical to the Step 7 baseline;
run b1_eval.py --on as a preview; ask the "Broker 4 ... 87%" question with and without
--version-aware. Save everything under runs/step09-10-filter/ and commit.
```

You check:

```
!cd track-b-kafka && RAG_CORPUS=simple_rag/corpus/kafka RAG_COLLECTION=kafka .venv/bin/python
simple_rag/b1_eval.py --on --out /tmp/b1check | tail -6
```

Expect: `0/84 = 0.0%`, superseded 9/9, exact_string 7/7, multi_hop 3/3. The same command with `--off` still gives `48.8%`.

Step 11: AFTER run

```
Confirm simple_rag/golden_kafka.jsonl still has sha1 850c67f2eab21aa1a6afabd481e5c3efccb7e0ba,
then run b1_eval.py --on (RAG_CORPUS=simple_rag/corpus/kafka RAG_COLLECTION=kafka), save the
log under runs/step11-after/ and commit results/version_filter_on.json.
```

Expect: `0/84 = 0.0%`, superseded 9/9, exact_string 7/7, multi_hop 3/3.

Step 12: ablation table, fair latency and report

```
Copy b1-tracking/track-b/tools/source/step12/b1_eval.py and B1_REPORT.md into
track-b-kafka/simple_rag/ (the report goes into results/). Run b1_eval.py --latency 5 and
b1_eval.py --table, show me results/ablation_summary.md, and commit b1_eval.py and results/.
```

You check: `!cat track-b-kafka/simple_rag/results/ablation_summary.md | head -14` shows 48.8% → 0.0%, with exact-string 7/7 → 7/7.

5. GitHub: publish and get the code

Repository (public): https://github.com/shreenathacc22/L2-RAG-B1-track-b-kafka-version-aware

Branch	Contents
`main`	Overview and links
`track-b-kafka-version-aware`	The full project, one commit per step, with docs and rebuild tools in `b1-track-b/`
`rag-deployment`	The runnable version-aware RAG: `run.sh` + README

Track A: https://github.com/shreenathacc22/L2-RAG-B1-version-aware-retrieval (`main`).

Prompt to publish (the one used on 2026-10-18):

```
Publish Track B to a new public GitHub repo L2-RAG-B1-track-b-kafka-version-aware with
branches main (overview README), track-b-kafka-version-aware (full project + docs + tools)
and rag-deployment (runnable package with run.sh). Never publish .env, .env.example, raw run
logs, local paths or my email: rewrite commit emails to my GitHub noreply address first,
scan every file before pushing, and verify the public repo afterwards.
```

You check:

```
!gh repo view shreenathacc22/L2-RAG-B1-track-b-kafka-version-aware --json visibility,defaultBranchRef
!git ls-remote --heads https://github.com/shreenathacc22/L2-RAG-B1-track-b-kafka-version-aware.git
```

Expect: `"visibility":"PUBLIC"`, and three branches.

Get the code anywhere:

```
git clone -b rag-deployment https://github.com/shreenathacc22/L2-RAG-B1-track-b-kafka-version-
aware.git b1-rag-deployment
cd b1-rag-deployment && ./run.sh setup && ./run.sh ingest && ./run.sh eval
```

6. Check everything at any time

Full test plan: `b1-track-b/docs/B1_TrackB_Kafka_Test-Scenarios_v1_2026-10-18.pdf` has 30 copy-paste test cases with expected messages. To run all of them: `python3 b1-track-b/tools/tests/run_tests.py --shell zsh`.

Quick checks:

```
cd ~/Desktop/AI-Architect_L2_Aug_2026/project_10082026/track-b-kafka
source .venv/bin/activate        # or: source ../mai-practitioner-labs/.venv/bin/activate
python simple_rag/b1_check.py --no-trap                        # data: PASS
python simple_rag/b1_check.py --no-trap --chroma               # data + Chroma metadata: PASS
python simple_rag/b1_inspect_chroma.py                         # look inside Chroma
git log --oneline -6                                           # one commit per step
RAG_CORPUS=simple_rag/corpus/kafka RAG_COLLECTION=kafka python simple_rag/b1_eval.py --off
```

7. If something goes wrong

What you see	What to tell Claude, or do
Claude edits the golden set after Step 6	"Revert that: the golden set is frozen. Use git checkout 2cff45a -- simple_rag/golden_kafka.jsonl"
`b1_eval.py --off` is no longer 48.8% after Steps 8–10	"OFF must reproduce the BEFORE baseline exactly. Find what changed besides the filter."
`No module named pip`	Normal for a `uv` venv: use `uv pip install ...`
`filter ON is not built yet`	Expected until Steps 9–10 are done
`--chroma` says `no ['doc_title', ...] in Chroma metadata`	The collection was ingested before Step 8: re-run `rag.py ingest --formats md`
`--chroma` says `collection is empty`	Set `RAG_COLLECTION=kafka` and ingest first
A proxy call hangs for minutes	Press `Esc`, retry; Steps 3–12 don't need the proxy at all
Claude seems lost after a long session	`/clear`, then paste the "first prompt of every session" again